



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Cátedra: Sistemas Operativos Año:2026

Trabajos Prácticos 1ra. Parte

Alumno:

- Luciano de Jesus Navarro DNI:44683191
- Romero Enzo Fabian DNI :41948148

PRÁCTICO N° 1: PROCESOS Y ADMINISTRACIÓN DEL PROCESADOR

Proceso	Tiempo de Llegada	Tiempo de Ejecución
P1	0	7
P2	2	5
P3	3	2
P4	5	4

EJERCICIO 1

Considere el siguiente conjunto de procesos, con la duración de la ráfaga de CPU y el tiempo de llegada dados en milisegundos:

Para este conjunto de procesos, realice lo siguiente: Representa gráficamente la ejecución de estos procesos utilizando los siguientes algoritmos de planificación:

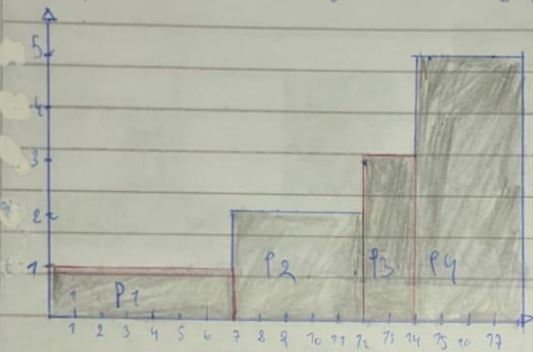
- FCFS (*First-Come, First-Served*)
- SJF (*Shortest Job First*) no apropiativo
- SRT (*Shortest Remaining Time*) apropiativo
- Round Robin (RR) con un cuanto (*quantum*) de 4 ms

Calcule el tiempo de espera promedio y el tiempo de retorno promedio para cada uno de los algoritmos mencionados.

Práctico 1 Sistema OperativoPlanificación de Primeros en entrar Primeros en salir (Fifo)

En muy simple, los procesos se ejecutan de acuerdo con su tiempo de llegada a la cola de lista.

Una vez que el proceso obtiene la CPU, se ejecuta hasta terminarlo, lo que en una disciplina no es preajustada.

Representación Gráfica de la ejecución mediante Fifo

2) Calcular Tiempo de espera Promedio y el Tiempo de Retorno Promedio Para cada uno de los ejercicios.

- Tiempo de espera de cada proceso: es el tiempo que el proceso ha estado en la cola de espera, inicio ejecución - tiempo de llegada
 $P_1 \text{ espera } 0 = 0 - 0 = 0$

$$P_2 \text{ espera } 5 \text{ ms} = 7 - 2 = 5 \text{ ms}$$

$$P_3 \text{ espera } 9 \text{ ms} = 12 - 3 = 9 \text{ ms}$$

$$P_4 \text{ espera } 9 \text{ ms} = 17 - 8 = 9 \text{ ms}$$

$$T_{EP} = T_{EP1} + T_{EP2} + T_{EP3} + T_{EP4}$$

CP

$$T_{EP} = \frac{0 + 5 + 9 + 9}{4} = \frac{23}{4} = 5.75 \text{ ms}$$

Tiempo de retorno: directamente al tiempo de ejecución.

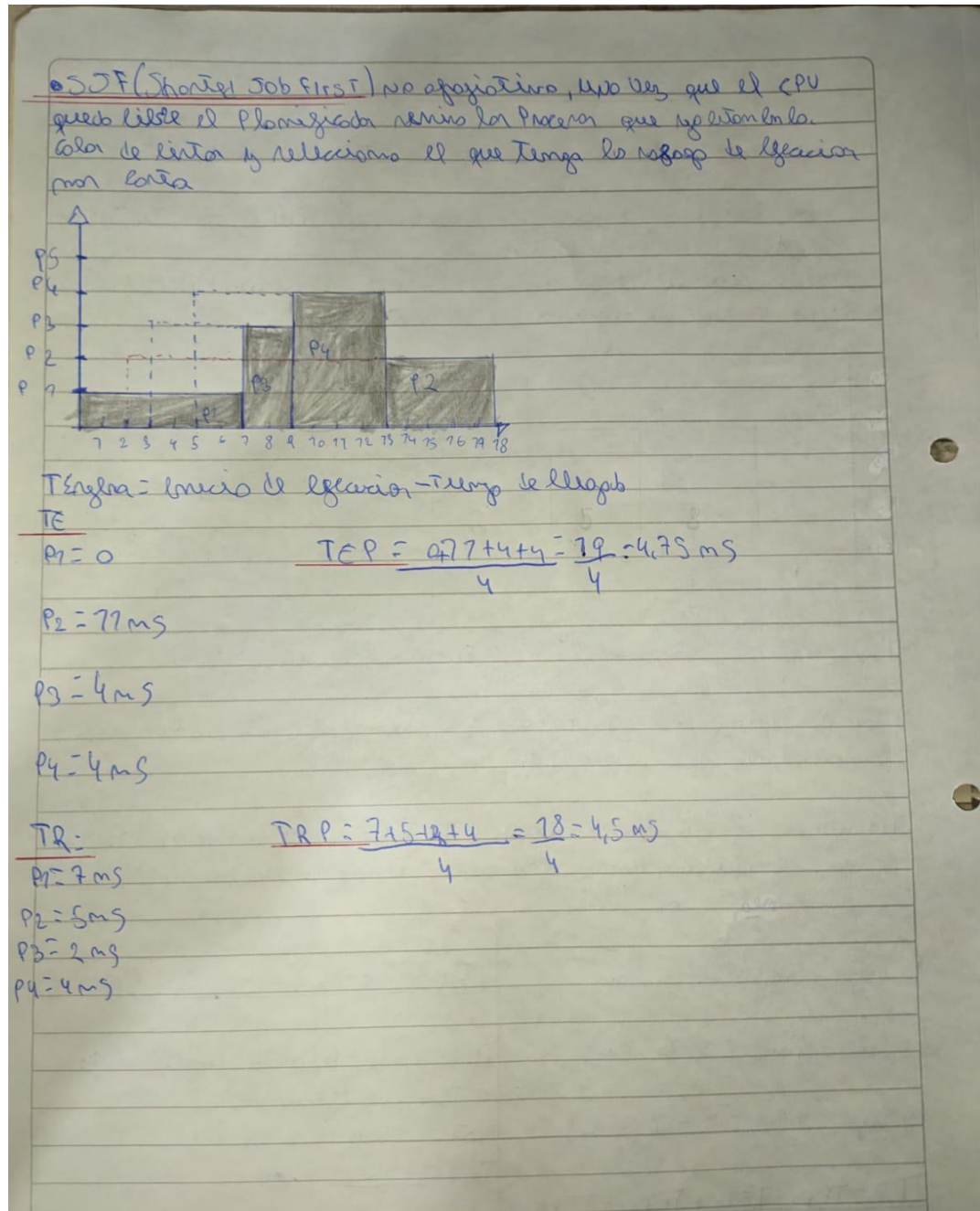
$$P_1 = 7, P_2 = 5, P_3 = 2, P_4 = 4$$

$$T_{RP} = \frac{7 + 5 + 2 + 4}{4} = 4.5 \text{ ms}$$

El tiempo de espera promedio es de 5,75 ms.

El tiempo de retorno promedio es de 4.5ms.

SJF (SHORTEST JOB FIRST) NO APROPIATIVO.

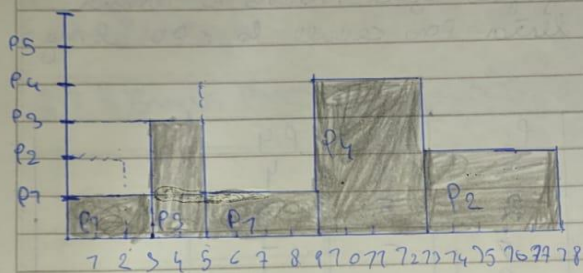


El tiempo de espera promedio es de 4.75ms.

El tiempo de retorno promedio es de 4.5 ms.

SRT (SHORTEST REMAINING TIME) APROPIATIVO.

SRT (Shortest Remaining Time) es la versión apropiativa de SJF se mantiene un proceso se ejecuta luego otro a la cola con un tiempo de ejecución menor al tiempo restante del que está en la CPU, el sistema le quita el CPU al proceso actual y se lo da al nuevo.



Tiempo de Espera: Fórmula suma de los tiempos en cada ronda

P1

Ronda 1: Ejecuto en 0 bytes 0 ms

Ronda 2: Resumido en 5 bytes 2 ms

$$TEP(P1) = \frac{0+2}{2} = 1 \text{ ms}$$

P2: Ejecuto en 73, no que interviene: Ejecuto 77 ms

$$TR = 77 = 7$$

P3: No bytes: 0 ms

$$P2 = 5$$

$$P3 = 2$$

$$P4 = 4$$

P4: Ejecuto 4 ms

$$TRP = \frac{7+7+0+4}{4} = 4.5 \text{ ms}$$

$$TEP = \frac{7+7+0+4}{4} = 4.5 \text{ ms}$$

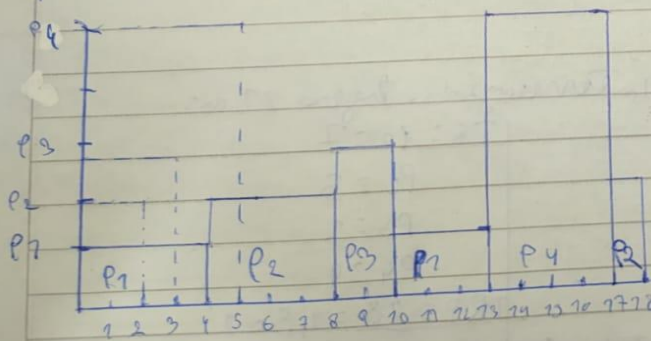
El tiempo de espera promedio es de 4 ms.

El tiempo de retorno promedio es de 4.5 ms

Round Robin es un algoritmo de Planificación diseñado para sistemas interactivos de tiempo compartido. Su objetivo es repartir equitativamente el uso del procesador entre todos los procesos de la cola de lista.

A cada proceso se le asigna un intervalo fijo de tiempo compartido llamado Cuanto (q). Si el proceso no termina dentro de su cuanto, la CPU le asigna a él el proceso se mueva al final de la cola de lista para cederle la CPU al siguiente.

	P1	P2	P3	P4
Ronda 1	4	4	2	4
TE	0	2	5	8
Ronda 2	3	7	—	—
TE	6	9	—	—
Ronda 3	—	—	—	—
TE	—	—	—	—
Tiempo espera	3	5.5	5	8
Tiempo Retorno	7	5	2	4
TE	$(3+5.5+5+8)/4 = 5.375 \text{ ms}$			
T.R.P	$(7+5+2+4)/4 = 4.5 \text{ ms}$			



El tiempo de espera promedio es de 5.375 ms.

El tiempo de retorno promedio es de 4.5 ms

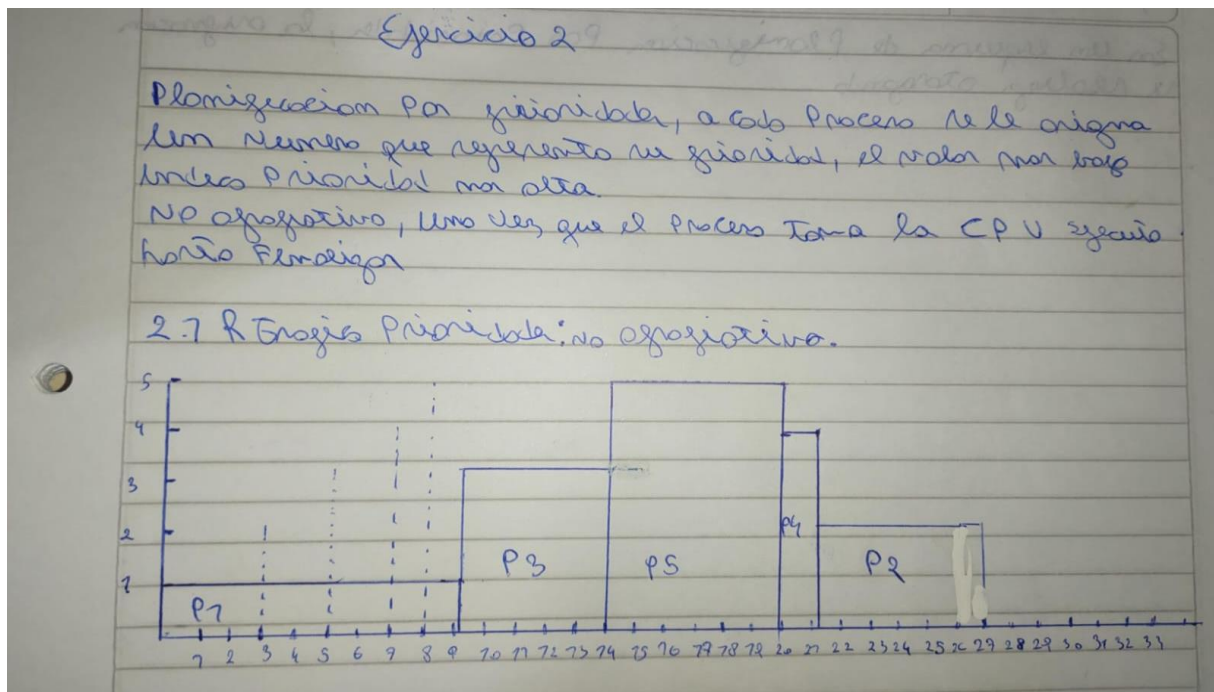
EJERCICIO 2

Se tiene un sistema que utiliza un algoritmo de planificación por prioridades. La siguiente tabla muestra un conjunto de procesos, su tiempo de ejecución, su tiempo de llegada y la prioridad asignada (un valor más bajo indica una prioridad más alta).

Proceso	Tiempo de Llegada	Tiempo de Ejecución	Prioridad
P1	0	9	2
P2	3	5	4
P3	5	6	1
P4	7	1	3
P5	8	6	1

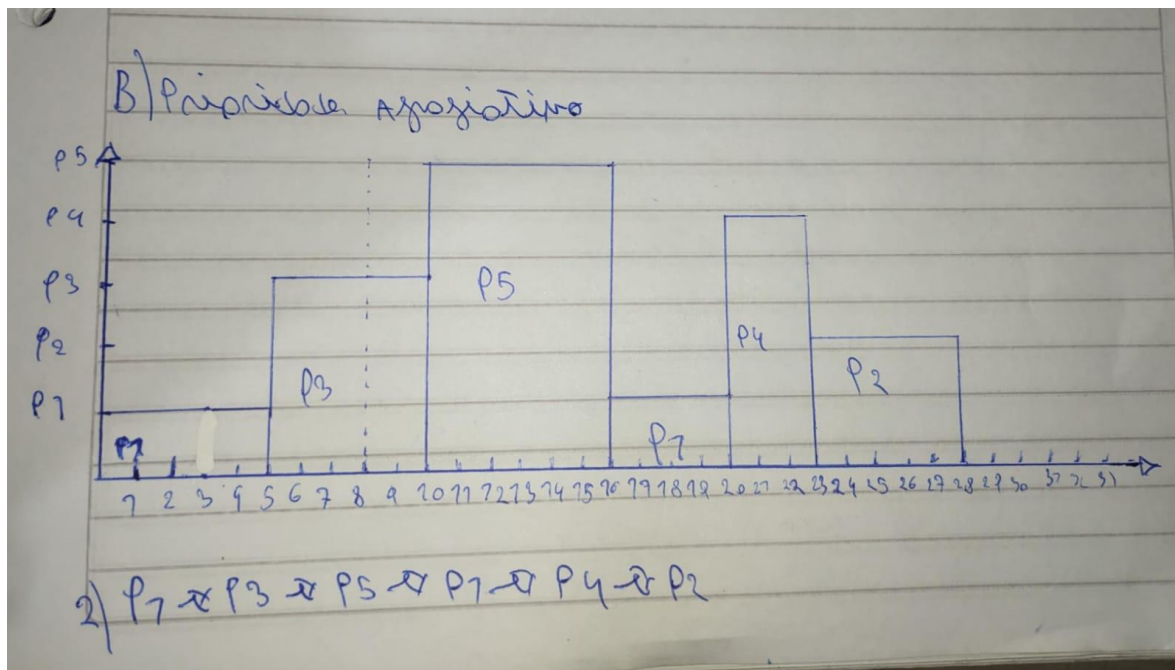
Representa gráficamente los siguientes algoritmos:

Prioridades no apropiativo:



P1 → P3 → P5 → P4 → P2

Prioridades apropiativo:



Secuencia de ejecución : $P_1 \rightarrow P_3 \rightarrow P_5 \rightarrow P_1 \rightarrow P_4 \rightarrow P_2$

. ¿Qué proceso sufre de mayor inanición (espera indefinida) en el esquema no apropiativo? Fundamente su respuesta basándose en los conceptos de bloqueo y asignación de CPU.

Respuesta a la Inanición (Starvation)

Proceso afectado: P2

Fundamentación Teórica: En un esquema de planificación por prioridades, la asignación de la CPU se realiza otorgando el procesador al proceso listo que posea la prioridad más alta (el valor numérico más bajo). En este sistema, P2 posee la prioridad más baja (Prioridad 4).

En el esquema no apropiativo, aunque un proceso que ya está en ejecución no puede ser interrumpido, cada vez que la CPU se libera, el planificador evalúa la cola de listos y selecciona al de mayor prioridad. Dado que P2 tiene la prioridad más relegada de todas, deberá esperar a que se completen todos los procesos con prioridades superiores (1, 2 y 3).

Si en un entorno real continúan ingresando nuevos procesos a la cola de listos con prioridades 1, 2 o 3, P2 nunca logrará que se le asignen los recursos de CPU, quedando relegado a una espera indefinida o inanición (starvation).

Ejercicio 3 Un sistema de tiempo compartido ha registrado la llegada de los siguientes procesos con sus respectivos tiempos de ejecución:

Proceso	Tiempo de Llegada	Tiempo de Ejecución
A	0	5
B	3	4
C	4	3
D	6	2

El administrador del sistema está evaluando el impacto del tamaño del cuanto en el rendimiento del sistema.

1. Aplique el algoritmo Round Robin (RR) y represente gráficamente para los siguientes tamaños de cuanto:

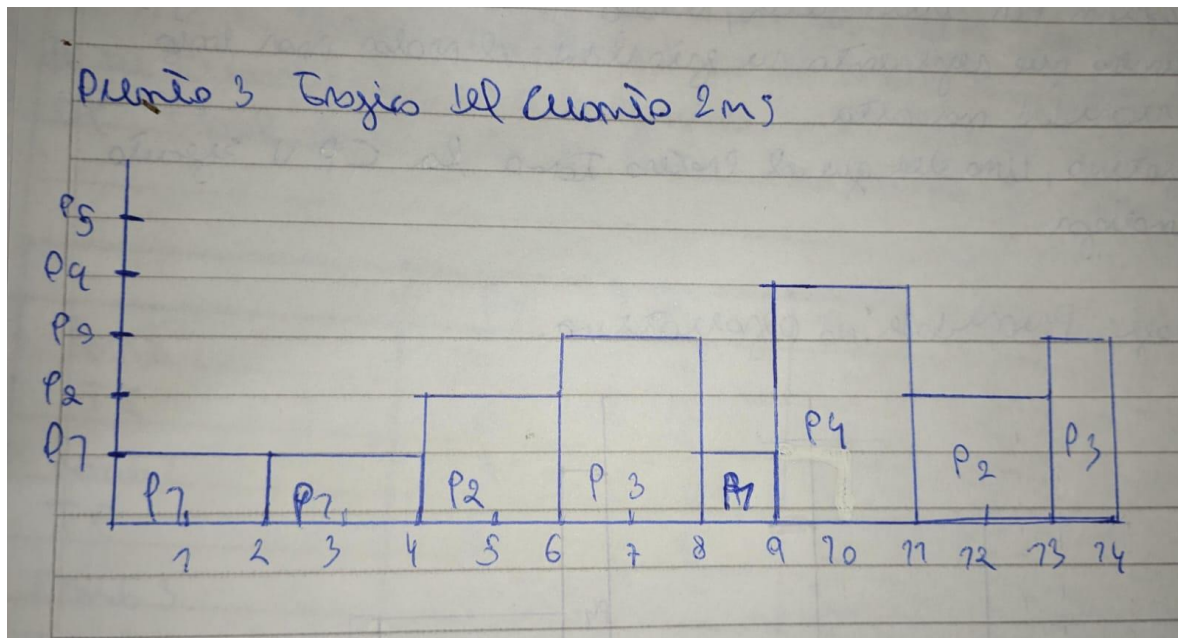
$q = 2$ ms

$q = 4$ ms

2. Calcule el tiempo de espera promedio para ambos valores de cuanto.
3. Analice y explique cómo la variación del tamaño del cuanto afecta la sobrecarga por cambio de contexto y el tiempo de respuesta del sistema, basándose en la teoría del tema.

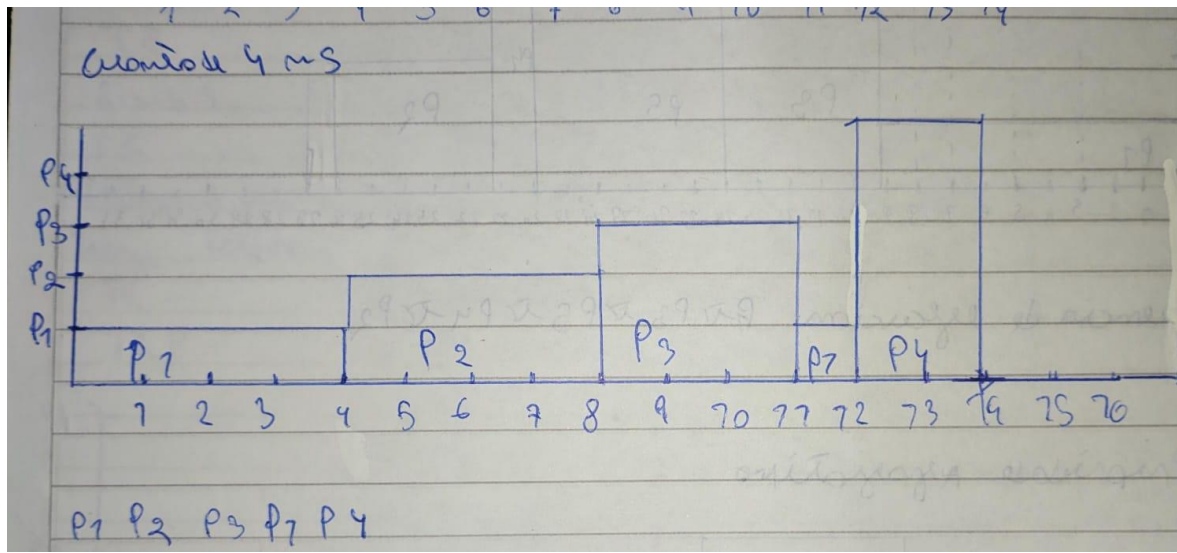
Cuanto de 2 ms:

	P1	P2	P3	P4
RONDA 1	2	2	2	2
TE	0	1	2	3
RONDA 2	2	2	1	-
TE	0	5	5	-
RONDA 3	1	-	-	-
TE	4	-	-	-
T. ESPERA	1,33	3	3,5	3
T. RETORNO	5	4	3	2
TEP	$(1,33 + 3 + 3,5 + 3) / 4 = 2,71 \text{ ms}$			
TRP	$(5 + 4 + 3 + 2) / 4 = 3,5 \text{ ms}$			



Cuanto de 4ms:

	P1	P2	P3	P4
RONDA 1	4	4	3	2
TE	0	1	4	6
RONDA 2	1	-	-	-
TE	7	-	-	-
RONDA 3	-	-	-	-
TE	-	-	-	-
T. ESPERA	3,5	1	4	6
T. RETORNO	5	4	3	2
TEP	$(3,5 + 1 + 4 + 6) / 4 = 3,625 \text{ ms}$			
TRP	$(5 + 4 + 3 + 2) / 4 = 3,5 \text{ ms}$			



3. Impacto del tamaño del cuanto El tamaño del cuanto determina qué tan seguido cambia la CPU de un proceso a otro:

- **Sobrecarga por cambio de contexto:** Si usamos un cuanto muy pequeño ($q = 2$ ms), la CPU pierde mucho tiempo guardando y cargando el estado de cada proceso para ir alternándolos. Al usar un cuanto más grande ($q = 4$ ms), hay menos interrupciones y la CPU aprovecha mejor el tiempo para ejecutar los programas.
- **Tiempo de respuesta:** Si el cuanto es pequeño, todos los procesos entran a la CPU rápidamente, dando la sensación de que el sistema responde más rápido a los usuarios. Si el cuanto es demasiado grande, los programas largos retienen la CPU mucho tiempo y el algoritmo termina funcionando como un FCFS (sin dar turno equitativo).

En resumen: Hay que buscar un equilibrio. El cuanto debe ser lo bastante grande para no desperdiciar tiempo en cambios de contexto, pero lo bastante chico para que el sistema siga sintiéndose rápido e interactivo.

Ejercicio 4: Transiciones de Estados de Procesos

a) Un proceso es admitido por el planificador de largo plazo, pero la memoria está llena. Finalmente, el planificador de corto plazo le asigna la CPU.

- **Secuencia de estados:**

Nuevo $\rightarrow$ Listo $\rightarrow$ Ejecución

- **Justificación:**

1. **Nuevo → Listo:** El planificador de largo plazo admite el proceso en el sistema. Aunque no haya memoria RAM disponible de inmediato, la estructura del proceso se crea y queda en la cola de listos (esperando su lugar en memoria).
2. **Listo → Ejecución:** Cuando la memoria se libera y el planificador de corto plazo le otorga la CPU, el proceso pasa a ejecutarse.

b) Un proceso en ejecución solicita una operación de Entrada/Salida (I/O) a un disco lento. Al completar la operación, es seleccionado inmediatamente para seguir.

- **Secuencia de estados:**

Ejecución → Bloqueado → Listo → Ejecución

- **Justificación:**

1. **Ejecución → Bloqueado:** Como el disco es lento, el proceso no puede continuar trabajando hasta tener los datos. Suelta la CPU para no desperdiciar tiempo y pasa a estar **Bloqueado**.
2. **Bloqueado → Listo:** Cuando el disco termina de leer los datos, el proceso ya puede continuar, por lo que vuelve a la cola de **Listos**. (*Importante: Un proceso NUNCA pasa de Bloqueado a Ejecución directamente*).
3. **Listo → Ejecución:** Como la consigna dice que es "seleccionado inmediatamente", el planificador le asigna la CPU sin demora.

c) Un proceso de alta prioridad se ejecuta en un sistema apropiativo y llega un proceso de máxima prioridad a la cola de listos.

- **Secuencia de estados:** (*Para el proceso que estaba ejecutando*)

Ejecución → Listo

- **Justificación:**

1. Al ser un sistema **apropiativo**, el sistema operativo le retira ("le saca a la fuerza") la CPU al proceso actual para dársela al recién llegado que tiene mayor prioridad.
2. El proceso interrumpido no cometió ningún error ni está esperando nada externo, por lo que simplemente vuelve a la cola de **Listos** a esperar que la CPU se libere de nuevo.

d) Un proceso en un sistema de tiempo compartido agota su cuanto de tiempo (quantum) y no ha terminado.

- **Secuencia de estados:**

Ejecución → Listo

- **Justificación:**

1. El reloj del sistema avisa que el tiempo máximo asignado (q) se terminó. El sistema operativo le retira el uso de la CPU.

2. Como al proceso todavía le falta trabajo por hacer, se lo reubica al final de la cola de **Listos** para que aguarde su próximo turno en la siguiente ronda.

Ejercicio 6: Análisis y Comparación de Planificadores

Responde a las siguientes preguntas conceptuales, justificando tus respuestas con la teoría del tema de procesos y administración del procesador

a) ¿Por qué el algoritmo SJF (Shortest Job First) es considerado óptimo en términos de tiempo de espera promedio, pero es difícil de implementar en la práctica en sistemas reales? El algoritmo SJF (Shortest Job First) es considerado óptimo en términos de tiempo de espera promedio porque siempre selecciona para ejecutarse el proceso que requiere menos tiempo de CPU. Esto minimiza el tiempo que los procesos tienen que esperar en la cola antes de ser ejecutados, ya que los procesos cortos se completan rápidamente, reduciendo así la suma total de los tiempos de espera de todos los procesos.

Sin embargo, es difícil de implementar en la práctica en sistemas reales por la siguiente razón:

- Para poder elegir el proceso más corto primero, el sistema necesita conocer de antemano el tiempo exacto que cada proceso tardará en ejecutarse.
- Puede provocar injusticia o “starvation” para procesos largos, que podrían quedarse esperando indefinidamente si constantemente llegan procesos más cortos.

b) Explica la diferencia fundamental entre el planificador de largo plazo (o planificador de trabajos) y el planificador de corto plazo (o planificador de CPU) . ¿Cuál de ellos tiene un impacto más directo en el grado de multiprogramación del sistema?

La diferencia fundamental entre el planificador de largo plazo (o de trabajos) y el planificador de corto plazo (o de CPU) radica en qué decisiones toman y con qué frecuencia operan:

Planificador de largo plazo (o de trabajos):

- Determina qué trabajos se admiten en el sistema para competir por los recursos.
- Opera muy poco frecuentemente (cada cierto tiempo), ya que decide sobre la entrada de nuevos procesos al sistema.
- Su principal objetivo es controlar el grado de multiprogramación, es decir, cuántos procesos estarán en memoria y listos para ejecutarse al mismo tiempo.

- Impacto directo en el grado de multiprogramación: alto, porque decide cuántos procesos pueden estar activos simultáneamente.

Planificador de corto plazo (o de CPU):

- Decide qué proceso listo se ejecuta a continuación en la CPU cuando ésta queda disponible.
- Opera muy frecuentemente, muchas veces por segundo, gestionando la ejecución de procesos de manera casi inmediata.
- Su principal objetivo es optimizar el uso de la CPU y mejorar la respuesta del sistema.
- Impacto en el grado de multiprogramación: bajo, porque no controla cuántos procesos hay en memoria, solo cuál se ejecuta primero.

El planificador de largo plazo (o de trabajos) tiene un impacto más directo en el grado de multiprogramación del sistema, porque es el que decide cuántos procesos se admiten en memoria y pueden competir por la CPU en un momento dado.

El planificador de corto plazo solo decide qué proceso de los que ya están en memoria se ejecuta primero, por lo que no afecta directamente la cantidad de procesos activos.

c) En un sistema con planificación por prioridades, ¿qué mecanismo se podría implementar para mitigar el problema de la inanición (starvation) de los procesos de baja prioridad? Describe brevemente cómo funcionaría.

Para mitigar la inanición (starvation) de los procesos de baja prioridad en un sistema con planificación por prioridades, se puede implementar el mecanismo de “envejecimiento” (aging).

Cómo funciona el envejecimiento:

- A medida que un proceso de baja prioridad espera en la cola de procesos listos, su prioridad se incrementa gradualmente con el tiempo.
- Eventualmente, la prioridad del proceso se vuelve lo suficientemente alta como para que sea seleccionado por el planificador y pueda ejecutarse.
- Esto garantiza que ningún proceso quede esperando indefinidamente, ya que todos los procesos aumentarán su prioridad lo suficiente como para acceder a la CPU eventualmente.

En pocas palabras, el envejecimiento “premia” la espera de los procesos menos prioritarios, evitando que siempre sean desplazados por procesos de mayor prioridad.

TRABAJO PRÁCTICO 2

1_ Mejor Ajuste

	12K	8K	15K	10K	20K	15K	8K	9K
5K		5K/3K						
10K				10K/0K				
5K							5K/3K	
8K								8K/1K
5K	5K/7K							

Primer Ajuste

	12K	8K	15K	10K	20K	15K	8K	9K
5K	5K/7K							
10K			10K/5K					
5K	5K/2K							
8K		8K/0K						
5K			5K/0K					

Peor Ajuste

	12K	8K	15K	10K	20K	15K	8K	9K
5K					5K/15K			
10K			10K/5K					
5K					5K/10K			
8K						8K/7K		
5K	5K/7K							

2_Primer Ajuste

	8K	20K	5K	20K	2K	10K	18K	45K
5K	5K/3K							
20K		20K/0K						
45K								45K/0K
20K				20K/0K				
2K	2K/1K							
18K							18K/0K	

Peor Ajuste

	8K	20K	5K	20K	2K	10K	18K	45K
5K								5K/40K
20K								20K/20K
45K								
20K		20K/0K						
2K				2K/18K				
18K								18K/2K

Mejor Ajuste

	8K	20K	5K	20K	2K	10K	18K	45K
5K			5K/0K					
20K		20K/0K						
45K								45K/0K
20K				20K/0K				
2K					2K/0K			
18K							18K/0K	

3_

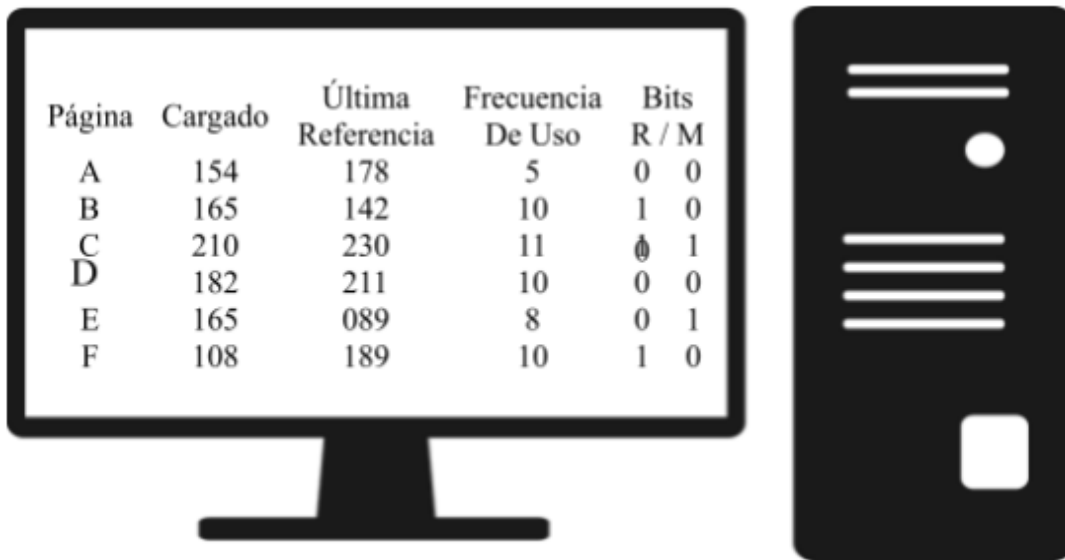
	2	1	6	Estado
6	2	1	6	N/F
4	4	2	1	F
6	6	4	2	F
5	5	6	4	F
8	8	5	6	F
8	8	5	6	N/F
3	3	8	5	F
4	4	3	8	F
1	1	4	3	F
2	2	1	4	F

a) Cantidad de marcos: 3 / Cantidad de páginas representadas: 7

b) Fallos: 8 / No Fallos: 2

c) Si se dispone de un marco de página más, se reducirá la cantidad de fallos

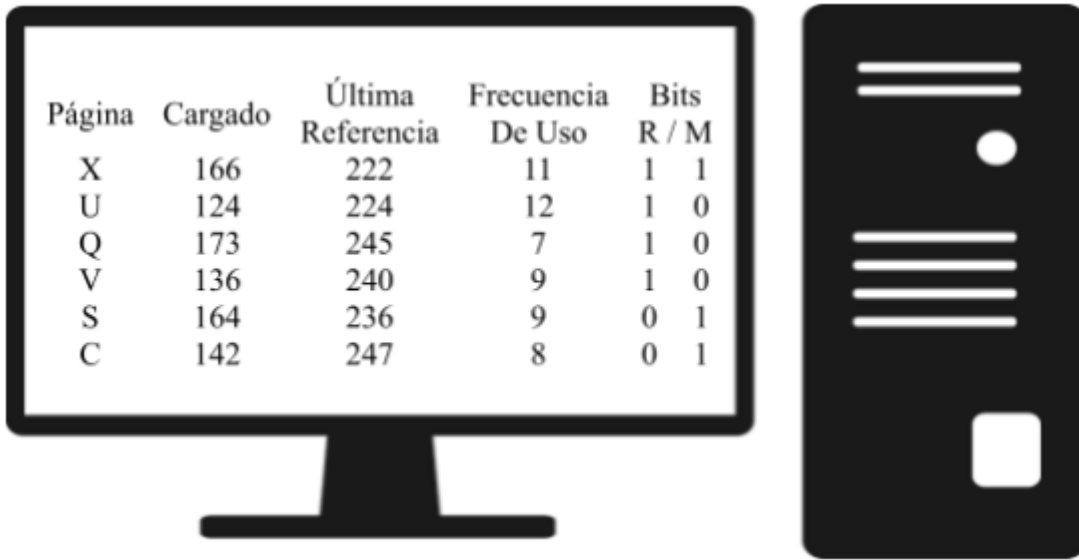
5_



Página	Cargado	Última Referencia	Frecuencia De Uso	Bits R / M	
A	154	178	5	0	0
B	165	142	10	1	0
C	210	230	11	0	1
D	182	211	10	0	0
E	165	089	8	0	1
F	108	189	10	1	0

- a) Por NRU, las páginas reemplazadas serían la A y la D respectivamente, al ser ambas las únicas en no ser referenciadas ni modificadas. En ese orden por el tiempo de carga, ya que, de las dos la A entró antes.
- b) Por FIFO, la primera página reemplazada sería la F al tener el menor tiempo de carga (o la que entró primero) y luego la A.
- c) Por LRU, la primera y segunda página serían la E y la B respectivamente, al ser las que han sido referenciadas hace más tiempo.
- d) Por LFU, las páginas reemplazadas son la A y la E, en ese orden, ya que son las usadas con menos frecuencia.

6_



Página	Cargado	Última Referencia	Frecuencia De Uso	Bits R / M	
X	166	222	11	1	1
U	124	224	12	1	0
Q	173	245	7	1	0
V	136	240	9	1	0
S	164	236	9	0	1
C	142	247	8	0	1

- a) Las páginas X y la U respectivamente, al ser las que no han sido usadas durante el mayor período de tiempo, aplicando la técnica LRU.
- b) La página U al tener el menor tiempo de carga (o la que entró primero) y luego la V, aplicando la técnica FIFO.
- c) Las páginas Q y la C, en ese orden, ya que son las usadas con menos frecuencia, aplicando la técnica LFU.
- d) Las páginas S y la C respectivamente, al ser las que no han sido cambiadas mientras estaban en el almacenamiento primario, aplicando la técnica NRU.

TRABAJO PRÁCTICO 3

1_a)

Archivo	Localización Inicial	Bloques Adicionales
A	B00	
B	B21	B12 – B18- B07
C	B23	B02 - B04 - B05
D	B03	B17- B01 - B08 - B16 - B11
E	B06	B09 - B13 - B19 - B14

NIL	B08	B04	B17	B05
A (1) B0	D (3) B1	C (2) B2	D (1) B3	C (3) B4
NIL	B09	NIL	B16	B13
C (4) B5	E (1) B6	B (4) B7	D (4) B8	E (2) B9
	NIL	B18	B19	NIL
LIBRE B10	D (6) B11	B (2) B12	E (3) B13	E (5) B14
	B11	B01	B07	B14
LIBRE B15	D (5) B16	D (2) B17	B (3) B18	E (4) B19
	B12		B02	
LIBRE B20	B (1) B21	LIBRE B22	C (1) B23	

b)

Archivo	Localización Inicial	Bloques Adicionales
A	0	
B	21	12 - 18 - 07
C	23	2 - 4 - 5
D	3	17- 1 - 8 - 16 - 11
E	6	9 - 13 - 19 - 14

0	NIL
1	8
2	4
3	17
4	5
5	NIL
6	9
7	NIL
8	16
9	13
10	
11	NIL
12	18
13	19
14	NIL
15	
16	11
17	1
18	7
19	14
20	
21	12
22	
23	2

2_I) Si eliminamos el bloque B se liberarían 4 bloques

NIL	B08	B04	B17	B05
A (1) B0	D (3) B1	C (2) B2	D (1) B3	C (3) B4
NIL	B09		B16	B13
C (4) B5	E (1) B6	LIBRE B7	D (4) B8	E (2) B9
	NIL		B19	NIL
LIBRE B10	D (6) B11	LIBRE B12	E (3) B13	E (5) B14
	B11	B01		B14
LIBRE B15	D (5) B16	D (2) B17	LIBRE B18	E (4) B19
			B02	
LIBRE B20	LIBRE B21	LIBRE B22	C (1) B23	

0	NIL
1	8
2	4
3	17
4	5
5	NIL
6	9
7	
8	16
9	13
10	
11	NIL
12	
13	19
14	NIL
15	
16	11
17	1
18	
19	14
20	
21	
22	
23	2

II) Si se agrega el archivo F de longitud 7 bloques físicos nos quedaría un bloque libre

NIL	B08	B04	B17	B05
A (1) B0	D (3) B1	C (2) B2	D (1) B3	C (3) B4
NIL	B09	B10	B16	B13
C (4) B5	E (1) B6	F (1) B7	D (4) B8	E (2) B9
B12	NIL	B15	B19	NIL
F (2) B10	D (6) B11	F (3) B12	E (3) B13	E (5) B14
B18	B11	B01	B20	B14
F (4) B15	D (5) B16	D (2) B17	F (5) B18	E (4) B19
B21	NIL		B02	
F (6) B20	F (7) B21	LIBRE B22	C (1) B23	

0	NIL
1	8
2	4
3	17
4	5
5	NIL
6	9
7	10
8	16
9	13
10	12
11	NIL
12	15
13	19
14	NIL
15	18
16	11
17	1
18	20
19	14
20	21
21	NIL
22	
23	2

III) No habría espacio disponible si el archivo E creciera en un tamaño de 5 bloques físicos.

5_a) Ruta de acceso absoluta para **cancion1.mp3** y **imagen.png**

cancion1.mp3: /home/pedro/musica/cancion1.mp3

imagen.png: /home/ana/descargas/imagen.png

b) Ruta de acceso relativa (desde /home/ana/) para **notas.txt** y **red.conf**

notas.txt: proyectos/notas.txt

red.conf: ../../etc/config/red.conf

6_a) Ruta de acceso absoluta para **deploy.sh** y **respaldo_2025.zip**

deploy.sh: /srv/diego/scripts/deploy.sh

respaldo_2025.zip: /srv/laura/backups/respaldo_2025.zip

b. Ruta de acceso relativa (desde /srv/laura/) para **estilos.css** y **compresor**

estilos.css: web/estilos.css

compresor: ../../bin/utilidades/compresor

7_

Archivo	Bloque Inicial	Longitud
M	B0	3
N	B6	2
O	B11	5

l) Utilizando asignación contigua, el archivo P1 de 4 bloques se implantaría en el único espacio disponible de B16 hasta B19

B1 M (1) B0	B2 M (2) B1	NIL M (3) B2	LIBRE B3	LIBRE B4
LIBRE B5	B7 N (1) B6	NIL N (2) B7	LIBRE B8	LIBRE B9
LIBRE B10	B12 O (1) B11	B13 O (2) B12	B14 O (3) B13	B15 O (4) B14
NIL O (5) B15	B17 P1 (1) B16	B18 P1 (2) B17	B19 P1 (3) B18	NIL P1 (4) B19

II) El archivo P2 de 5 bloques no podría implantarse ya que no quedaría espacio contiguo disponible.

8_

Archivo	Bloques de Datos (en orden lógico)
J1	B14 – B15 – B09
J2	B05 – B11 – B00 – B07 – B13
J3	B03 – B10

	B14 – B15 – B09	B05 – B11 – B00	
J2 (1) B0	J1 B1	J2 (1) B2	J3 B3
B07 – B13 – NIL		B03 – B10 – NIL	
J2 (2) B4	J2 (1) B5	J3 B6	J2 (2) B7
B8	J1 B9	J3 B10	J2 (1) B11
B12	J2 (2) B13	J1 B14	J1 B15

Practico 4 parte teorica

1. Capas del Software de Entrada/Salida

Capas del Software de E/S (desde el hardware hacia el usuario):

Manejadores de Interrupciones (Interrupt Handlers): Es la capa más baja. Su función principal es ocultar las interrupciones del hardware al resto del sistema operativo, suspendiendo momentáneamente el proceso actual cuando un dispositivo notifica que completó su tarea.

Controladores de Dispositivos (Device Drivers): Son programas específicos para cada hardware. Su función es traducir los comandos genéricos del sistema operativo en instrucciones específicas que el controlador del dispositivo físico entienda.

Software del SO independiente del dispositivo: Maneja funciones generales comunes a todos los dispositivos, como el nombrado uniforme, la protección, la asignación de buffers (buffering) y la gestión de bloques/errores.

Software de E/S en espacio de usuario: Son las bibliotecas de código (como `stdio.h` en C) y las aplicaciones. Permite a los programas realizar llamadas al sistema (system calls) para solicitar operaciones de lectura o escritura de forma sencilla.

Beneficio de la arquitectura en capas:

Aporta modularidad e independencia del hardware. Permite reemplazar o actualizar un dispositivo físico (o su driver) sin necesidad de reescribir todo el sistema operativo ni modificar los programas del usuario.

2. Dispositivos de E/S y Controladores

Diferencia entre Dispositivo de Bloque y Dispositivo de Carácter:

Dispositivo de Bloque: Almacena la información en bloques de tamaño fijo (cada uno con su propia dirección). Se puede leer o escribir cada bloque de forma independiente.

Ejemplos: Discos Rígidos (HDD), Unidades de Estado Sólido (SSD), Pendrives USB.

Dispositivo de Carácter: Envía o recibe la información como un flujo continuo de caracteres (octetos), sin una estructura de bloques fija y sin posibilidad de posicionarse (unseekable).

Ejemplos: Teclado, Ratón (mouse), Impresora, Puerto Serie.

Papel del Controlador de Dispositivo:

El controlador (componente electrónico/hardware del dispositivo, como un chip o placa) actúa como intermediario directo entre la CPU y el dispositivo mecánico o físico. Convierte el flujo de bits en serie que envía el dispositivo en un bloque de bytes, valida que no haya errores de transmisión y expone registros de memoria que la CPU puede leer o escribir para enviarle comandos.

3. Spooling y Buffering

Explicación de Conceptos:

Buffering (Almacenamiento en Memoria Intermedia): Consiste en guardar temporalmente datos en la memoria RAM (buffer) durante la transferencia entre dos dispositivos o entre un programa y un hardware.

Problema que resuelve: Soluciona la diferencia de velocidades entre el procesador (muy rápido) y los dispositivos (muy lentos), y adapta diferentes tamaños de transferencia de datos.

Spooling (Simultaneous Peripheral Operations On-Line): Es una técnica que utiliza el disco como una "cola de espera" o almacén intermedio para interceptar las salidas que van dirigidas a un dispositivo que no puede ser compartido simultáneamente.

Problema que resuelve: Soluciona la exclusividad de los dispositivos de acceso exclusivo (como las impresoras). Evita que un programa quede totalmente bloqueado mientras espera que la impresora termine de imprimir el documento de otro usuario.

Diferencia Fundamental:

El buffering gestiona datos directamente en la memoria RAM para suavizar diferencias de velocidad durante una transferencia de un único proceso. El spooling utiliza el disco para gestionar colas de trabajos completas de múltiples procesos que compiten por un mismo dispositivo exclusivo.

Tp 4 parte practica

Si en un determinado momento se presentan solicitudes de lectura de bloques en los cilindros: 28, 10, 30, 8 y 24. Transcurrido 5 milisegundos se presentan nuevas solicitudes para los cilindros 12, 22, 25, 8, 10 y 20. ¿Cuál será la secuencia de búsqueda y el número de movimientos si se utilizan los siguientes algoritmos considerando el brazo en el cilindro 19 al principio?

SOLICITUD: 28, 10, 30, 8, 24 // 12, 22, 25, 8, 10, 20

INICIO: 19

FCFS (PRIMERO EN LLEGAR, PRIMERO EN SALIR)

SECUENCIA: 19, 28, 10, 30, 8, 24, 12, 22, 25, 8, 10, 20

MOVIMIENTOS: $9 + 18 + 20 + 22 + 16 + 12 + 10 + 3 + 17 + 2 + 10 = 139$

SSTF (MENOR TIEMPO DE BÚSQUEDA PRIMERO)

SECUENCIA: 19, 24, 28, 30, 10, 8, 10, 12, 20, 22, 25

MOVIMIENTOS: $5 + 4 + 2 + 20 + 2 + 2 + 2 + 8 + 2 + 3 = 50$

SSTF es el mas optimo ya que atiende toas las solicitudes recorriendo solo 50 movimientos , en comparación con los 139 que requiere fcfs.

2.

Si en un determinado momento se presentan solicitudes de lectura de bloques en los cilindros: 15, 9, 13, 20, 8, 13 y 20. Transcurrido 5 milisegundos se presentan nuevas solicitudes para los cilindros 15, 18, 18, 9, 9, 17, 12, 5, 4, 13, 21 y 18. ¿Cuál será la secuencia de búsqueda y el número de movimientos si se utilizan los siguientes algoritmos considerando el brazo en el cilindro 12 al principio?

SOLICITUD: 15, 9, 13, 20, 8, 13, 20 // 15, 18, 18, 9, 9, 17, 12, 5, 4, 13, 21, 18

INICIO: 12

FCFS (PRIMERO EN LLEGAR, PRIMERO EN SALIR)

SECUENCIA: 12, 15, 9, 13, 20, 8, 13, 20, 15, 18, 18, 9, 9, 17, 12, 5, 4, 13, 21, 18

MOVIMIENTOS: $3 + 6 + 4 + 7 + 12 + 5 + 7 + 5 + 3 + 0 + 9 + 0 + 8 + 5 + 7 + 1 + 9 + 8 + 3 = 100$

SSTF (MENOR TIEMPO DE BÚSQUEDA PRIMERO)

SECUENCIA: 12, 13, 13, 15, 9, 8, 9, 9, 12, 13, 15, 17, 18, 18, 18, 20, 20, 21, 5, 4

MOVIMIENTOS: $1 + 0 + 2 + 6 + 1 + 1 + 0 + 3 + 1 + 2 + 2 + 1 + 0 + 0 + 2 + 0 + 1 + 16 + 1 = 40$

SSTF es el más óptimo por requerir la menor cantidad de movimientos.

3) Si en un determinado momento se presentan solicitudes de lectura de bloques en los cilindros: 30, 3, 4, 5, 24, 12, 2, 26 y 18. Transcurrido 3 milisegundos se presentan nuevas solicitudes para los cilindros 54, 34, 12, 6, 22, 17, 3 y 6. ¿Cuál será la secuencia de búsqueda y el número de movimientos si se utilizan los siguientes algoritmos considerando el brazo en el cilindro 2 al principio?

SOLICITUD: 30, 3, 4, 5, 24, 12, 2, 26, 18 // 54, 34, 12, 6, 22, 17, 3, 6

INICIO: 2 FCFS (PRIMERO EN LLEGAR, PRIMERO EN SALIR)

SECUENCIA: 2, 30, 3, 4, 5, 24, 12, 2, 26, 18, 54, 34, 12, 6, 22, 17, 3, 6

MOVIMIENTOS: $28 + 27 + 1 + 1 + 19 + 12 + 10 + 24 + 8 + 36 + 20 + 22 + 6 + 16 + 5 + 14 + 3 = 252$

SSTF (MENOR TIEMPO DE BÚSQUEDA PRIMERO)

SECUENCIA: 2, 2, 3, 4, 5, 12, 18, 24, 26, 30, 3, 6, 6, 12, 17, 22, 34, 54

MOVIMIENTOS: $0 + 1 + 1 + 1 + 7 + 6 + 6 + 2 + 4 + 27 + 3 + 0 + 6 + 5 + 5 + 12 + 20 = 106$

SSTF es el más óptimo por requerir la menor cantidad de movimientos.

Si en un determinado momento se presentan solicitudes de lectura de bloques en los cilindros: 20, 15, 14, 5, 32, 9, 12, 9, 52, 34 y 27.

Transcurrido 7 milisegundos se presentan nuevas solicitudes para los cilindros 4, 5, 2, 33, 5, 19, 4, 18, 5, 12, 10 y 15. ¿Cuál será la secuencia de búsqueda y el número de movimientos si se utilizan los siguientes algoritmos considerando el brazo en el cilindro 6 al principio?

Ejercicio 4

SOLICITUD: 20, 15, 14, 5, 32, 9, 12, 9, 52, 34, 27 // 4, 5, 2, 33, 5, 19, 4, 18, 5, 12, 10, 15

INICIO: 6

FCFS (PRIMERO EN LLEGAR, PRIMERO EN SALIR)

SECUENCIA: 6, 20, 15, 14, 5, 32, 9, 12, 9, 52, 34, 27, 4, 5, 2, 33, 5, 19, 4, 18, 5, 12, 10, 15

MOVIMIENTOS: $14 + 5 + 1 + 9 + 27 + 23 + 3 + 3 + 43 + 18 + 7 + 23$
 $+ 1 + 3 + 31 + 28 + 14 + 15 + 14 + 13 + 7 + 2 + 5 = 309$

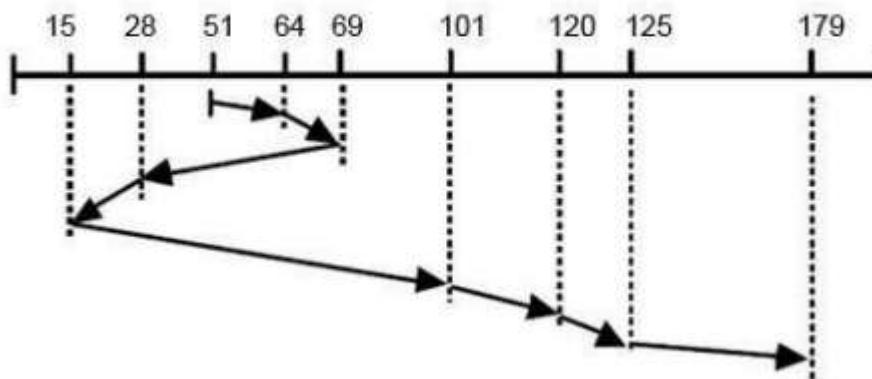
SSTF (MENOR TIEMPO DE BÚSQUEDA PRIMERO)

SECUENCIA: 6, 5, 9, 9, 12, 14, 15, 20, 27, 32, 34, 52, 33, 19, 18, 15,
12, 10, 5, 5, 5, 4, 4, 2

MOVIMIENTOS: $1 + 4 + 0 + 3 + 2 + 1 + 5 + 7 + 5 + 2 + 18 + 19 + 14$
 $+ 1 + 3 + 3 + 2 + 5 + 0 + 0 + 1 + 0 + 2 = 98$

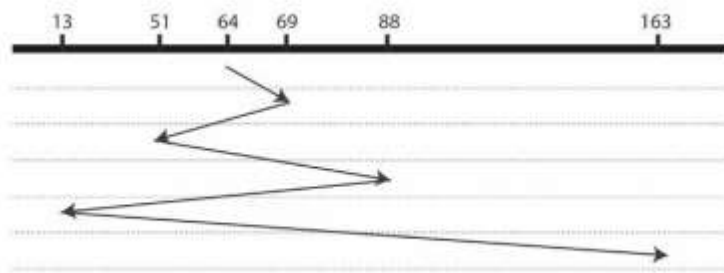
SSTF es el más óptimo por requerir la menor cantidad de movimientos.

5)INDIQUE LA TECNICA UTILIZADA



La técnica utilizada es SSTF (Shortest Seek Time First), ya que el cabezal siempre se mueve hacia la solicitud más cercana a su posición actual. En este método no importa el orden en que llegaron los datos, porque el criterio de selección es la distancia mínima entre el cabezal y las solicitudes pendientes.

7. Indique la técnica utilizada:



La técnica utilizada es SSTF (Shortest Seek Time First), ya que el cabezal siempre se mueve hacia la solicitud más cercana a su posición actual. En este método no importa el orden en que llegaron los datos, porque el criterio de selección es la distancia mínima entre el cabezal y las solicitudes pendientes